

Polynomial-Time Reductions

Lemma 34.3, NP-Completeness, and Theorem 34.4

1 Why Reductions Matter

Central Idea

A polynomial-time reduction is a formal way to say:

“If I can solve problem L_2 efficiently, then I can solve L_1 efficiently.”

Thus $L_1 \leq_p L_2$ means L_1 is no harder than L_2 , up to polynomial-time translation.

Reductions let us compare computational difficulty without needing to solve either problem directly. They are the main tool used to define and prove **NP**-complete.

2 Polynomial-Time Many-One Reductions

Definition

Let $L_1, L_2 \subseteq \{0, 1\}^*$ be languages. We say that

$$L_1 \leq_p L_2$$

if there exists a polynomial-time computable function

$$f : \{0, 1\}^* \rightarrow \{0, 1\}^*$$

such that for every string $x \in \{0, 1\}^*$,

$$x \in L_1 \iff f(x) \in L_2.$$

The function f transforms instances of L_1 into instances of L_2 while preserving the answer.

How to Read $L_1 \leq_p L_2$

- L_1 reduces to L_2 .
- A solver for L_2 can be used to solve L_1 .
- L_2 is at least as hard as L_1 .
- The direction matters: to prove L_2 is hard, reduce a known hard problem **to** L_2 .

3 Lemma 34.3

Lemma 1 (Reduction Preserves Polynomial-Time Solvability). *If $L_1, L_2 \subseteq \{0, 1\}^*$ are languages such that*

$$L_1 \leq_p L_2,$$

then

$$L_2 \in \mathbf{P} \implies L_1 \in \mathbf{P}.$$

Proof

Let A_2 be a polynomial-time algorithm deciding L_2 . Let F be a polynomial-time reduction algorithm computing the reduction function f .

We construct an algorithm A_1 for L_1 :

1. On input x , compute $f(x)$ using F .
2. Run A_2 on $f(x)$.
3. Output whatever answer A_2 outputs.

Correctness follows directly from the reduction condition:

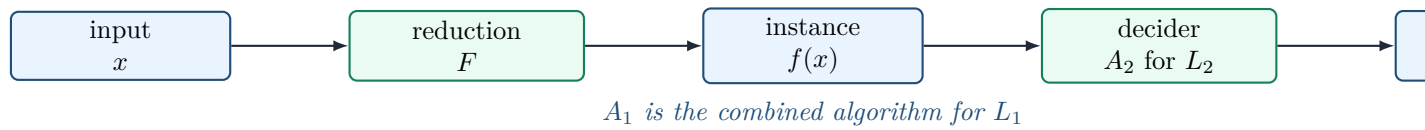
$$x \in L_1 \iff f(x) \in L_2.$$

Therefore A_1 accepts exactly the strings in L_1 .

The running time is polynomial because:

- F runs in polynomial time;
- F outputs only polynomially many bits;
- A_2 runs in polynomial time on that output.

Thus A_1 is a polynomial-time decider for L_1 , so $L_1 \in \mathbf{P}$.



4 NP-Completeness

Definition: NP-Complete

A language $L \subseteq \{0, 1\}^*$ is **NP-complete** if:

1. $L \in \mathbf{NP}$;
2. for every language $L' \in \mathbf{NP}$,

$$L' \leq_p L.$$

The class of all **NP-complete** languages is denoted by

NPC.

Definition: NP-Hard

A language or problem L is **NP-hard** if every language in **NP** reduces to it:

$$\forall L' \in \mathbf{NP}, \quad L' \leq_p L.$$

Unlike **NP-complete** problems, an **NP-hard** problem is not required to be in **NP**.

Class	Requirement	Meaning
NP	Has polynomial-time verifiable YES certificates.	Solutions can be checked efficiently.
NP-hard	Every problem in NP reduces to it.	At least as hard as all problems in NP .
NP-complete	In NP and NP-hard .	The hardest problems inside NP .

Why NP-Complete Problems Matter

If one **NP**-complete problem has a polynomial-time algorithm, then every problem in **NP** has a polynomial-time algorithm. Therefore one efficient algorithm for one **NP**-complete problem would settle the entire **P** versus **NP** question.

5 Theorem 34.4

Theorem 1 (NP-Complete Problem in **P** Implies $\mathbf{P} = \mathbf{NP}$). *If any NP-complete problem is polynomial-time solvable, then*

$$\mathbf{P} = \mathbf{NP}.$$

*Equivalently, if any problem in **NP** is not polynomial-time solvable, then no **NP**-complete problem is polynomial-time solvable.*

Proof

Suppose L is **NP**-complete and $L \in \mathbf{P}$.

Let L' be any language in **NP**. Since L is **NP**-complete, every language in **NP** reduces to L . Therefore

$$L' \leq_p L.$$

Because $L \in \mathbf{P}$, Lemma 34.3 implies

$$L' \in \mathbf{P}.$$

Since L' was an arbitrary language in **NP**, we have shown

$$\mathbf{NP} \subseteq \mathbf{P}.$$

We always know

$$\mathbf{P} \subseteq \mathbf{NP},$$

because any problem that can be solved in polynomial time can also be verified in polynomial time. Hence

$$\mathbf{P} = \mathbf{NP}.$$

The second statement is the contrapositive: if there exists some language in **NP** that is not in **P**, then no **NP**-complete language can be in **P**.

6 How to Prove a Problem Is NP-Complete

Standard Proof Template

To prove a language B is **NP**-complete:

1. Prove $B \in \mathbf{NP}$ by giving a polynomial-time verifier.
2. Choose a known **NP**-complete language A .
3. Construct a polynomial-time reduction

$$A \leq_p B.$$

4. Prove the reduction preserves yes/no answers:

$$x \in A \iff f(x) \in B.$$

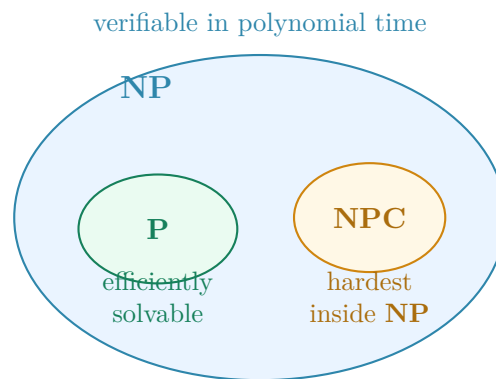
Common Mistake

To prove B is hard, the reduction must go from a known hard problem to B :

$$\text{known } \mathbf{NP}\text{-complete problem } A \leq_p B.$$

Showing $B \leq_p A$ only proves that B is no harder than A .

7 Relationship Between P, NP, and NPC



This diagram shows the standard belief that

$$\mathbf{P} \neq \mathbf{NP}.$$

Under that belief, **NP**-complete problems lie inside **NP** but outside **P**.

Current State of Knowledge

No one has proved whether $\mathbf{P} = \mathbf{NP}$ or $\mathbf{P} \neq \mathbf{NP}$. No polynomial-time algorithm is known for any **NP**-complete problem, and no proof rules one out.

8 Key Ideas

Idea	Meaning
$L_1 \leq_p L_2$	Convert instances of L_1 into equivalent instances of L_2 in polynomial time.
Lemma 34.3	If $L_1 \leq_p L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.
NP-hard	Every NP language reduces to the problem.
NP-complete	The problem is both in NP and NP-hard .
Theorem 34.4	One polynomial-time algorithm for one NP -complete problem implies $\mathbf{P} = \mathbf{NP}$.
Contrapositive	If $\mathbf{P} \neq \mathbf{NP}$, then no NP -complete problem is polynomial-time solvable.

9 Quick Self-Check

Review Questions

1. What does $L_1 \leq_p L_2$ mean?
2. Why does $L_1 \leq_p L_2$ make L_2 at least as hard as L_1 ?
3. In Lemma 34.3, how is the algorithm for L_1 constructed?
4. What two properties define an **NP**-complete language?
5. What is the difference between **NP**-complete and **NP**-hard?
6. Why would a polynomial-time algorithm for one **NP**-complete problem imply $\mathbf{P} = \mathbf{NP}$?
7. When proving a new problem is **NP**-complete, which direction should the reduction go?